WIRESHARK PACKET ANALYSIS

1.Introduction

Wireshark is a network protocol analyzer used to capture and examine network traffic in real time.Wireshark is extremely helpful in helping users understand how packets actually move through the network in real time-It's especially helpful for understanding commonly used networking protocols like-TCP,DNS or HTTP.By analyzing these packets users can see how connections are established(TCP three way handshake),How DNS queries are resolved etc.

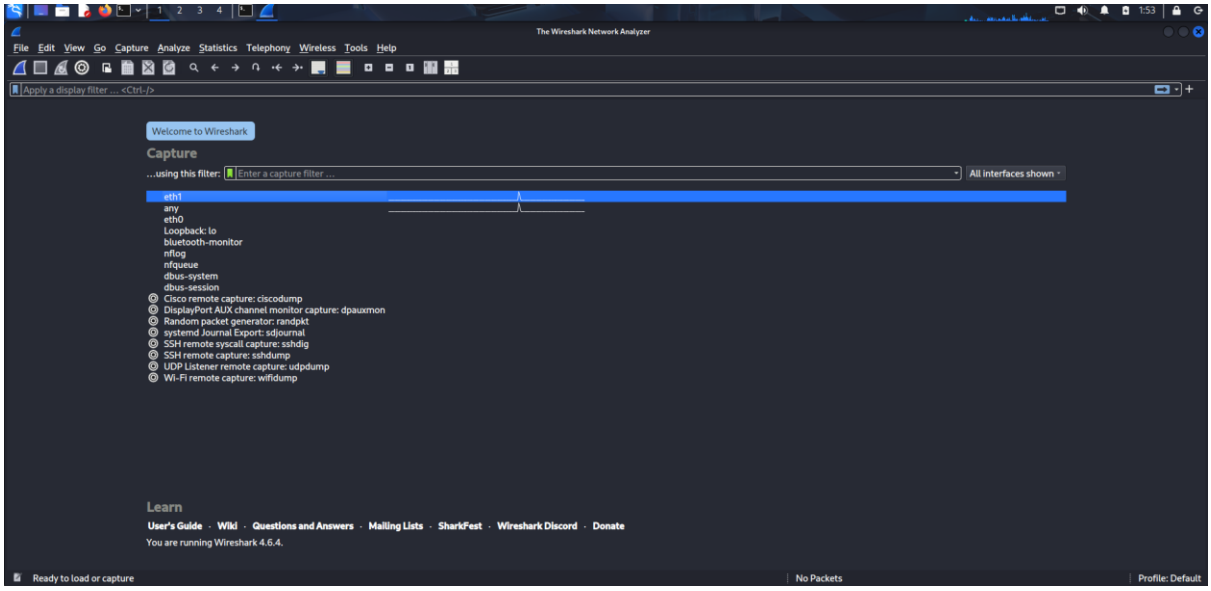


Figure 1:Wireshark home screen

This is the wireshark home screen showing all the available interfaces. From this window we can select through which interface we want to monitor network traffic.

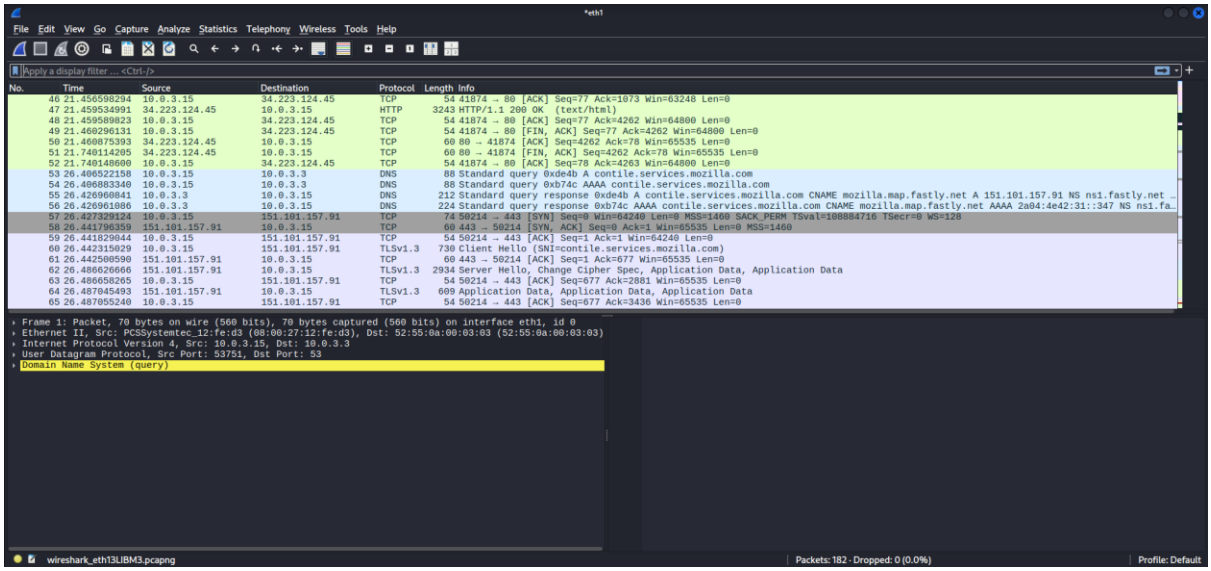


Figure 2:Captured packets

Figure 2 shows the packet capture screen in Wireshark after a few minutes of network activity, including browsing, pinging websites etc. This list displays captures with source IP address, destination IP address etc and for a more in depth view we can left click on individual packets and view a detailed breakdown on the bottom left pane.

2.Packet Analysis

2.1 DNS

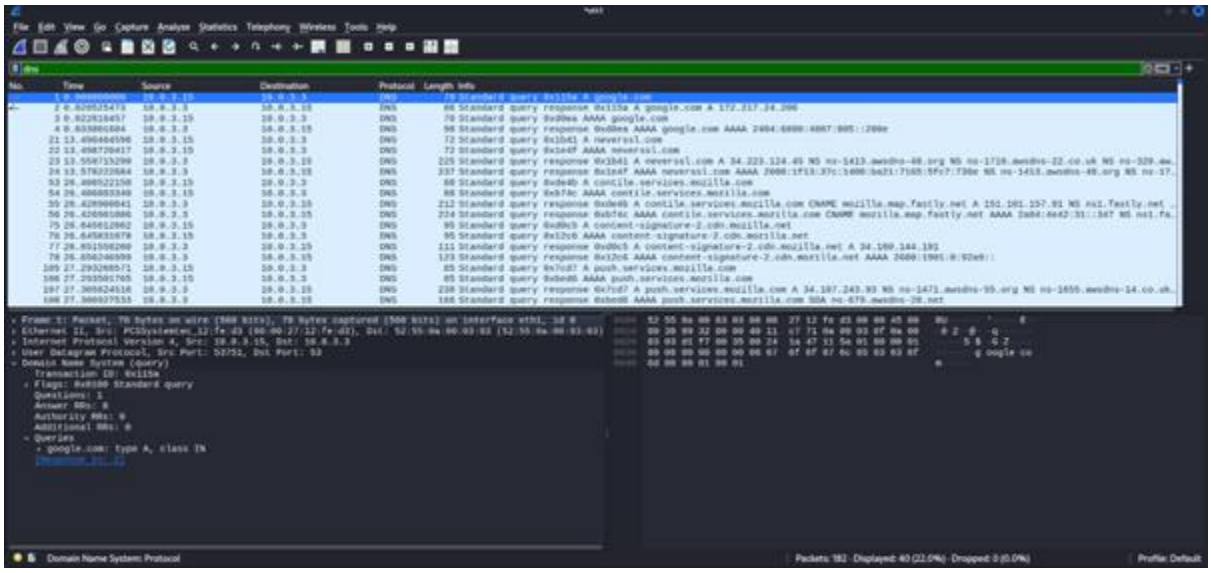


Figure 3:DNS Packet Filtered View

Figure 3 shows a filtered view of only DNS packets by using the DNS filter. By applying this filter, only packets related to the **Domain Name System (DNS)** are displayed, allowing easier analysis of DNS traffic. The packet list shows multiple DNS queries and responses generated when accessing websites. Each entry contains details such as the source and destination IP addresses, packet length, and query information.

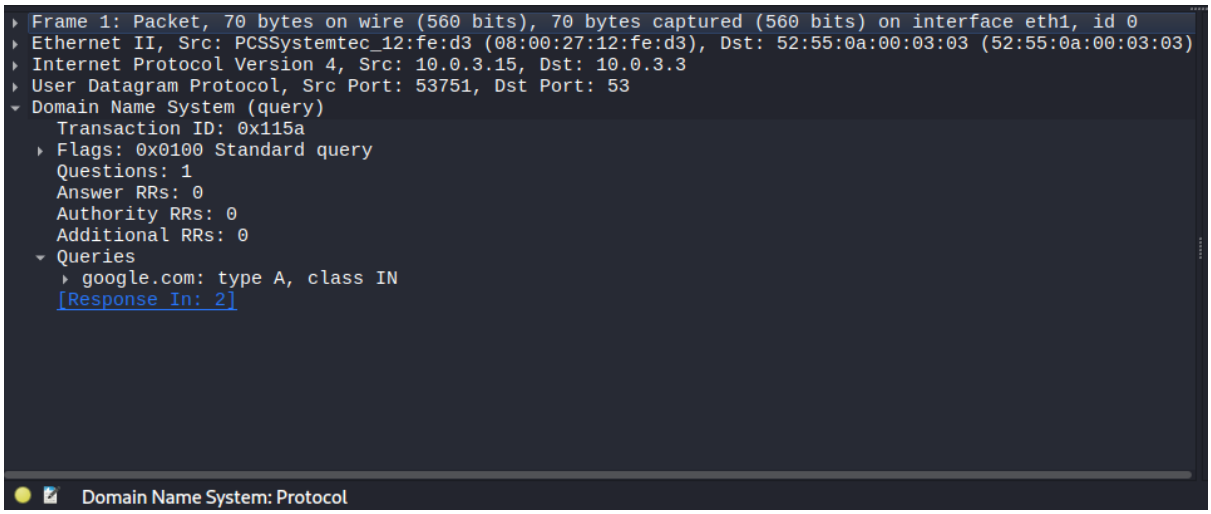


Figure 4:Detailed DNS Packet Inspection

Figure 4 shows the detailed analysis of a selected DNS packet. The packet details pane displays the different protocol layers involved in the communication, including Ethernet, Internet Protocol (IP), User Datagram Protocol (UDP), and DNS. The DNS section shows information such as the transaction ID, query type, flags, and the domain name being queried (google.com).

2.2 ICMP

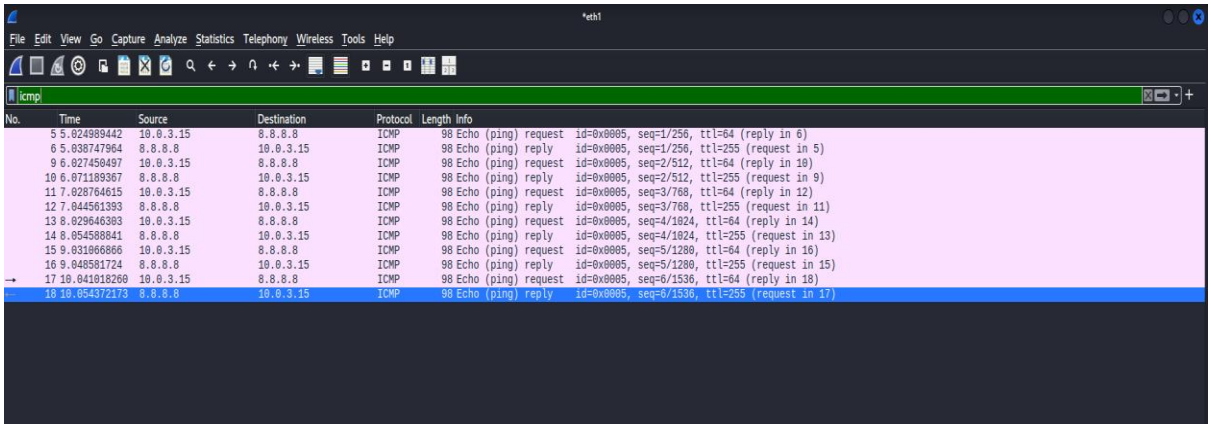


Figure 5:ICMP Ping Packet Capture

Figure 5 shows ICMP packets captured in Wireshark after applying the ICMP filter. The packets represent ping requests and replies exchanged between the local system and the destination IP address 8.8.8.8 (Google DNS server). The packet list displays details such as the source IP address, destination IP address, protocol type (ICMP), packet length, sequence numbers, and Time To Live (TTL) values. These packets demonstrate how the ping command tests network connectivity by sending ICMP Echo Requests and receiving Echo Replies, confirming that communication between the devices is successful.

<u>NOTE: The source IP address is 10.0.3.15</u>

2.3 HTTP

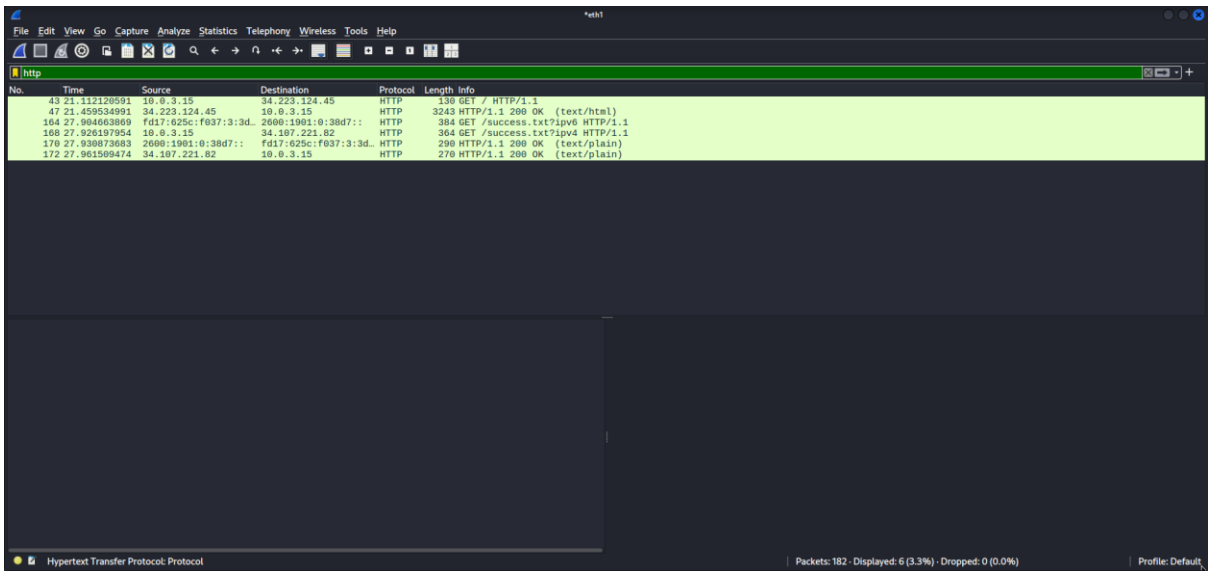


Figure 6:Filtered HTTP Packets

Figure 6 shows HTTP packets after applying the HTTP filter in Wireshark. The captured packets include HTTP requests and responses generated while accessing a website. The packet list shows communication between the local system (10.0.3.15) and the remote server (34.223.124.45) using the HTTP protocol over port 80. Entries such as GET requests and OK responses indicate that the client requested web resources and the server successfully returned the requested content.

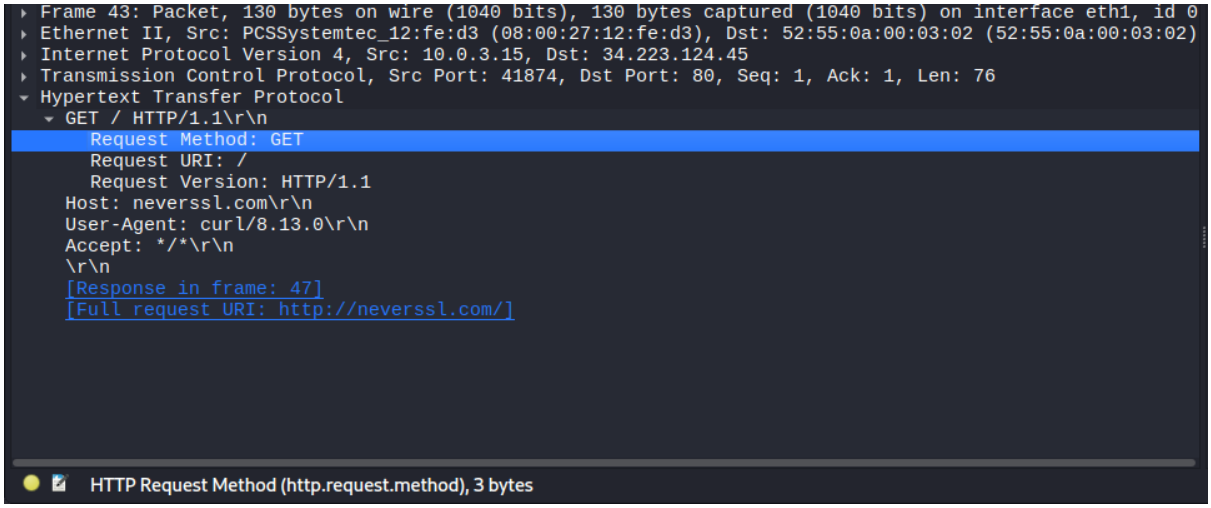


Figure 7:Detailed HTTP Request Packet Inspection

Figure 7 displays the detailed breakdown of a selected HTTP packet. The HTTP section reveals important information such as the request method (GET), request URI, host name (neverssl.com), HTTP version, it also specifies exactly which frame the response for this GET request is provided (frame 47). This helps in understanding how a client sends a request to retrieve a web page from a server.

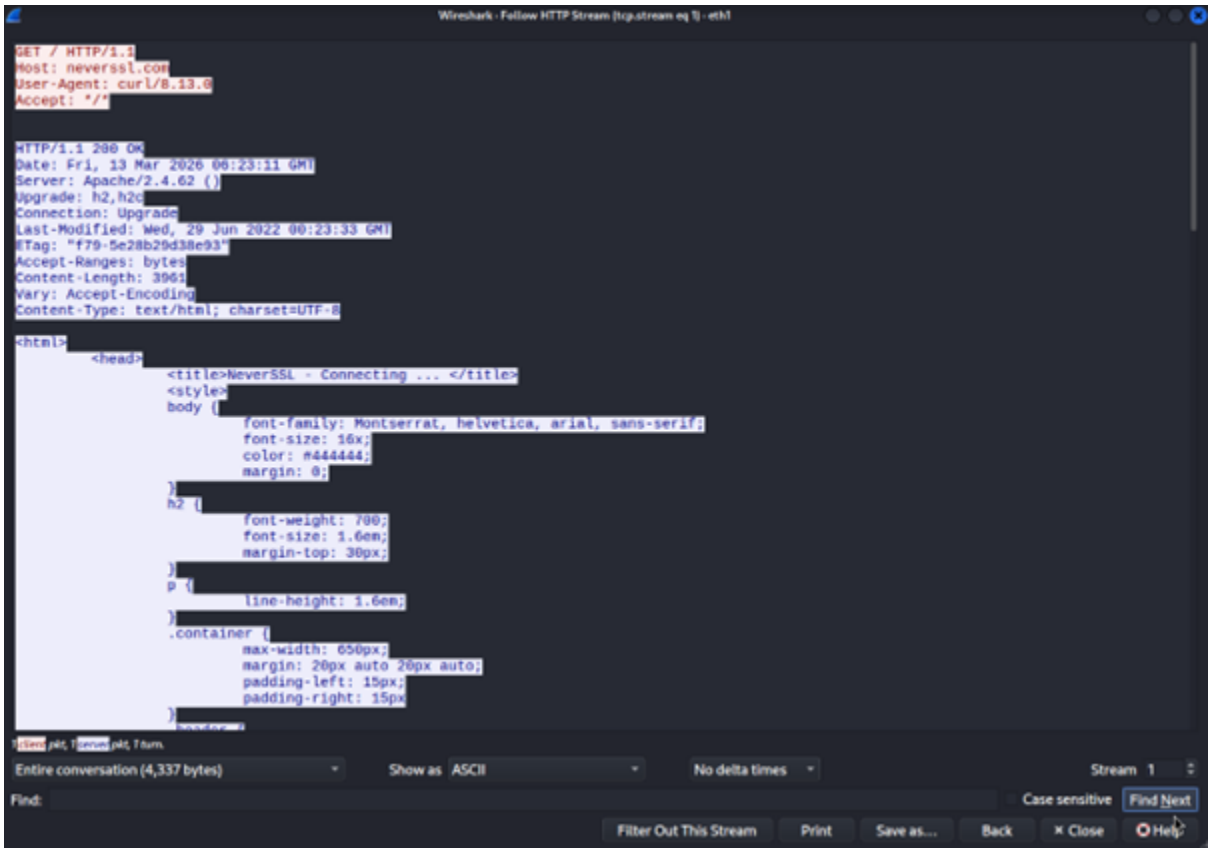


Figure 8:Follow TCP Stream Output

If we right click on a specific packet and hover over follow on click on follow TCP stream we get this output. This is a reconstruction of the communication between server and client in readable form. Figure 8 shows the GET request and the OK response followed by the HTML content of the webpage.

Security Analysis

The main difference between HTTP and HTTPS is that in HTTPS the content would be shown in an encrypted format using TLS. Wireshark would show the TLS handshake process but the data would be in encrypted packets.

Plaintext communication is a security risk because the data is not encrypted, meaning anyone who intercepts the network traffic can easily read the information being transmitted, tools such as Wireshark can display the full contents of the communication, including webpage data, usernames, passwords, cookies, and other sensitive information.

2.4 TCP

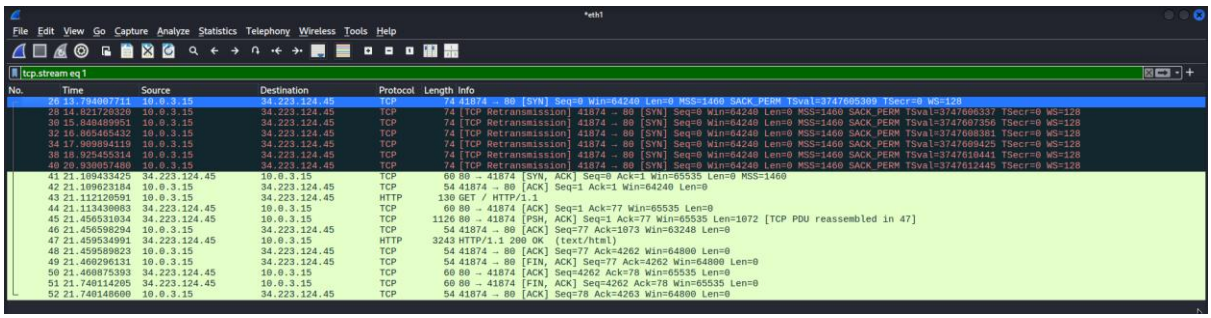
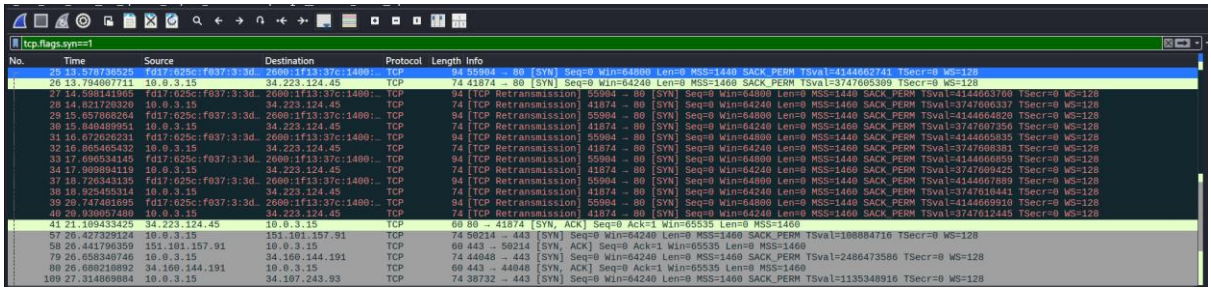


Figure 9:TCP Stream Packets

Figure 9 shows the TCP packets for the same HTTP packet we followed the TCP frame for. Here is where we can clearly see how connections are established via the three way handshake. The source(10.0.3.15) sends an initial SYN packet to the destination(34.223.124.45) as the 26th frame. The SYN-ACK is not sent from the destination until the 41st frame so we get to see TCP retransmission in play, the source keeps retransmitting the SYN packet as it is not receiving an acknowledgement. At the 42nd frame the source sends the ACK packet and hence connection is established after which the HTTP GET request is made.

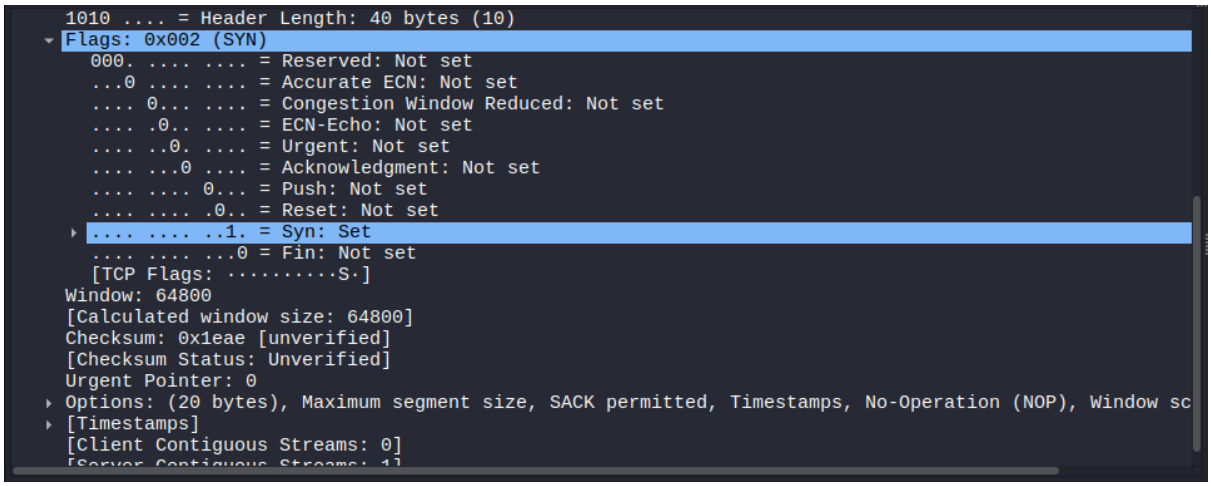


tcp.flags.syn==1

No.	Time	Source	Destination	Protocol	Length	Info
25	13.578736525	fd17:625c:f037:3:3d…	2600:1f13:37c:1400:…	TCP	94	55904 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4144662741 TSecr=0 WS=128
26	13.794007711	10.0.3.15	34.223.124.45	TCP	74	41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747605309 TSecr=0 WS=128
27	14.598141965	fd17:625c:f037:3:3d…	2600:1f13:37c:1400:…	TCP	94	[TCP Retransmission] 55904 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4144663760 TSecr=0 WS=128
28	14.821720320	10.0.3.15	34.223.124.45	TCP	74	[TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747606337 TSecr=0 WS=128
29	15.657868264	fd17:625c:f037:3:3d…	2600:1f13:37c:1400:…	TCP	94	[TCP Retransmission] 55904 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4144664820 TSecr=0 WS=128
30	15.840489951	10.0.3.15	34.223.124.45	TCP	74	[TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747607356 TSecr=0 WS=128
31	16.672626231	fd17:625c:f037:3:3d…	2600:1f13:37c:1400:…	TCP	94	[TCP Retransmission] 55904 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4144665835 TSecr=0 WS=128
32	16.865465432	10.0.3.15	34.223.124.45	TCP	74	[TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747608381 TSecr=0 WS=128
33	17.696534145	fd17:625c:f037:3:3d…	2600:1f13:37c:1400:…	TCP	94	[TCP Retransmission] 55904 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4144666859 TSecr=0 WS=128
34	17.909894119	10.0.3.15	34.223.124.45	TCP	74	[TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747609425 TSecr=0 WS=128
37	18.726343135	fd17:625c:f037:3:3d…	2600:1f13:37c:1400:…	TCP	94	[TCP Retransmission] 55904 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4144667889 TSecr=0 WS=128
38	18.925455314	10.0.3.15	34.223.124.45	TCP	74	[TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747610441 TSecr=0 WS=128
39	20.747481695	fd17:625c:f037:3:3d…	2600:1f13:37c:1400:…	TCP	94	[TCP Retransmission] 55904 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4144669910 TSecr=0 WS=128
40	20.930057480	10.0.3.15	34.223.124.45	TCP	74	[TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747612445 TSecr=0 WS=128
41	21.109433425	34.223.124.45	10.0.3.15	TCP	60	80 → 41874 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460
57	26.427329124	10.0.3.15	151.101.157.91	TCP	74	50214 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=108884716 TSecr=0 WS=128
58	26.441796359	151.101.157.91	10.0.3.15	TCP	60	443 → 50214 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460
79	26.658340746	10.0.3.15	34.160.144.191	TCP	74	44048 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2486473586 TSecr=0 WS=128
80	26.680210892	34.160.144.191	10.0.3.15	TCP	60	443 → 44048 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460
109	27.314869884	10.0.3.15	34.107.243.93	TCP	74	38732 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1135348916 TSecr=0 WS=128

Figure 10:TCP SYN Packets

Figure 10 shows TCP packets filtered using the tcp.flags.syn==1 filter, which displays packets used to initiate TCP connections. The highlighted packets represent SYN packets sent by the client to start communication with the server



```
    1010 .... = Header Length: 40 bytes (10)
  ▾ Flags: 0x002 (SYN)
      000. .... .... = Reserved: Not set
      ...0 .... .... = Accurate ECN: Not set
      .... 0... .... = Congestion Window Reduced: Not set
      .... .0.. .... = ECN-Echo: Not set
      .... ..0. .... = Urgent: Not set
      .... ...0 .... = Acknowledgment: Not set
      .... .... 0... = Push: Not set
      .... .... .0.. = Reset: Not set
    ▸ .... .... ..1. = Syn: Set
      .... .... ...0 = Fin: Not set
      [TCP Flags: ··********S·]
    Window: 64800
    [Calculated window size: 64800]
    Checksum: 0x1eae [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  ▸ Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP), Window sc
  ▸ [Timestamps]
    [Client Contiguous Streams: 0]
    [Server Contiguous Streams: 1]
```

Figure 11:Detailed TCP SYN Flag Inspection

Figure 11 displays the detailed analysis of a TCP SYN packet in the packet details pane. The TCP header shows that the SYN (Synchronize) flag is set, indicating that the packet is part of the TCP three-way handshake used to establish a connection. Additional details such as the TCP window size, checksum, and TCP options are also visible, which help manage data flow and reliability during communication.

3.Conclusion

This experiment demonstrated how Wireshark can be used to capture and analyze network traffic in real time. By examining different types of packets such as **DNS, ICMP, HTTP, and TCP**, it was possible to observe how network protocols operate and interact during normal communication. The analysis also showed how tools like packet filters and stream analysis

help in understanding protocol behavior and troubleshooting network activity. Overall, Wireshark provides valuable insight into how data flows across a network and is an essential tool for network monitoring, troubleshooting, and cybersecurity analysis.